

Node.js Event Loop - Complete Interview Guide

Table of Contents

1. [Understanding JavaScript Basics](#)
 2. [Node.js Runtime](#)
 3. [The Event Loop](#)
 4. [Code Examples](#)
 5. [Interview Questions & Answers](#)
-

Understanding JavaScript Basics

What is JavaScript at its core?

JavaScript is a **synchronous, blocking, single-threaded** language. Let me break this down:

1. Synchronous

Meaning: Code executes line by line, from top to bottom.

Example:

```
function A() {  
  console.log("A");  
}  
  
function B() {  
  console.log("B");  
}  
  
A();  
B();  
  
// Output:  
// A  
// B
```

Function A runs completely first, then function B runs. Simple!

2. Blocking

Meaning: If one task takes time, everything else waits. Nothing else can happen until the current task finishes.

Example:

```
function A() {  
  // Imagine this takes 5 seconds  
  for(let i = 0; i < 5000000000; i++) {  
    // Heavy computation  
  }  
  console.log("A finished");  
}  
  
function B() {  
  console.log("B");  
}  
  
A(); // This will block everything  
B(); // B has to wait until A is done
```

Real-world problem: Ever seen a website freeze? That's blocking! The browser can't do anything else until the heavy task completes.

3. Single-threaded

Meaning: JavaScript has only ONE main thread to execute all code.

Think of it like a restaurant with only ONE chef:

- The chef can cook only ONE dish at a time
- Other orders must wait in line
- No parallel cooking happening

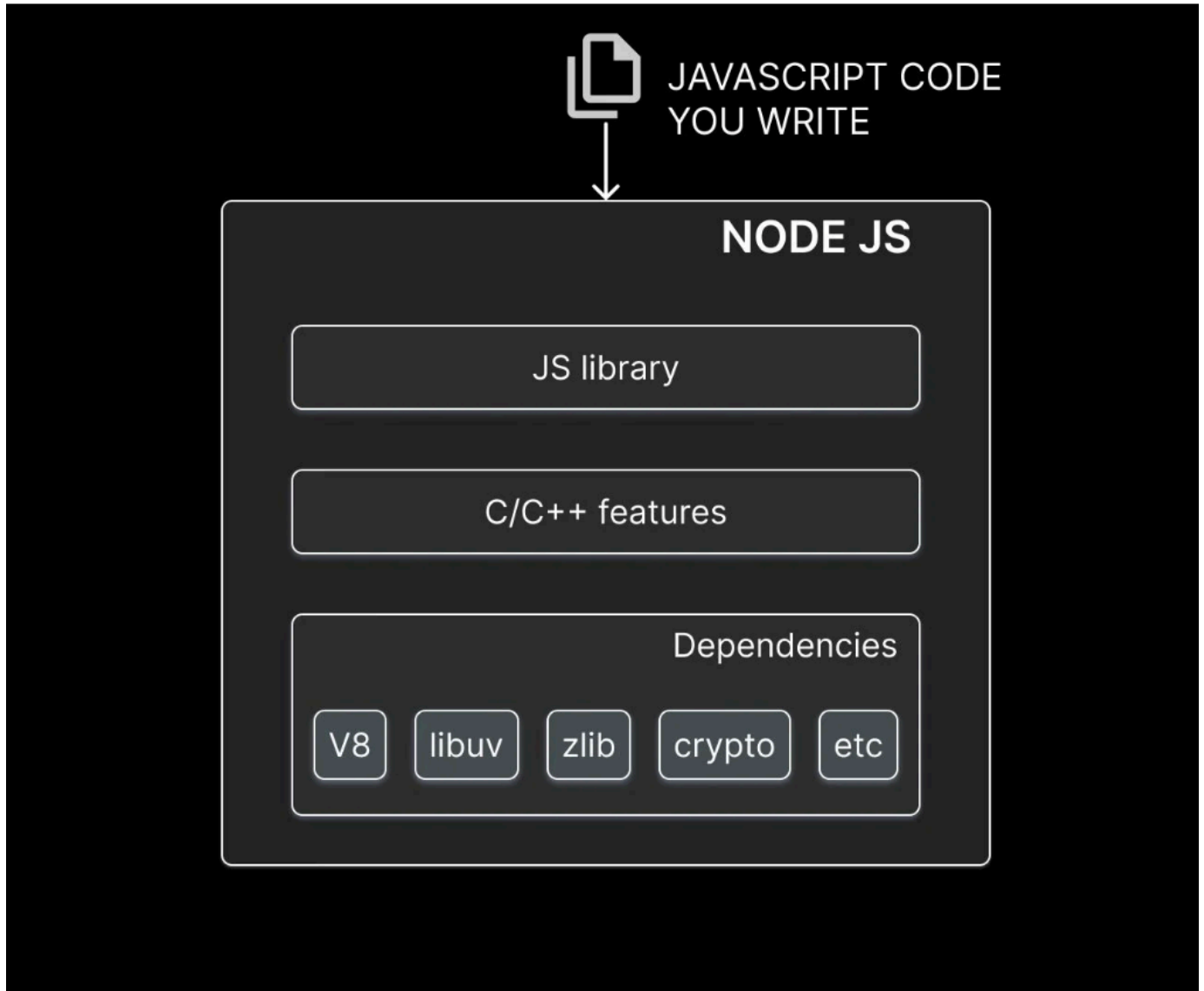
The Problem: If we need to fetch data from a database (takes 3 seconds), our entire program waits for 3 seconds doing nothing! This is terrible for performance.

The Solution: Node.js gives us asynchronous capabilities!

Node.js Runtime

What is Node.js Runtime?

Node.js runtime is an environment that lets you run JavaScript outside the browser. It has 3 main parts:



Understanding Libuv

Libuv is the hero of asynchronous programming in Node.js!

- Written in C language
- Cross-platform (works on Windows, Mac, Linux)
- Provides the Event Loop
- Manages thread pool for heavy tasks

How it works:

1. When you write async code (like reading a file)
2. Libuv takes that task away from JavaScript
3. JavaScript continues executing other code

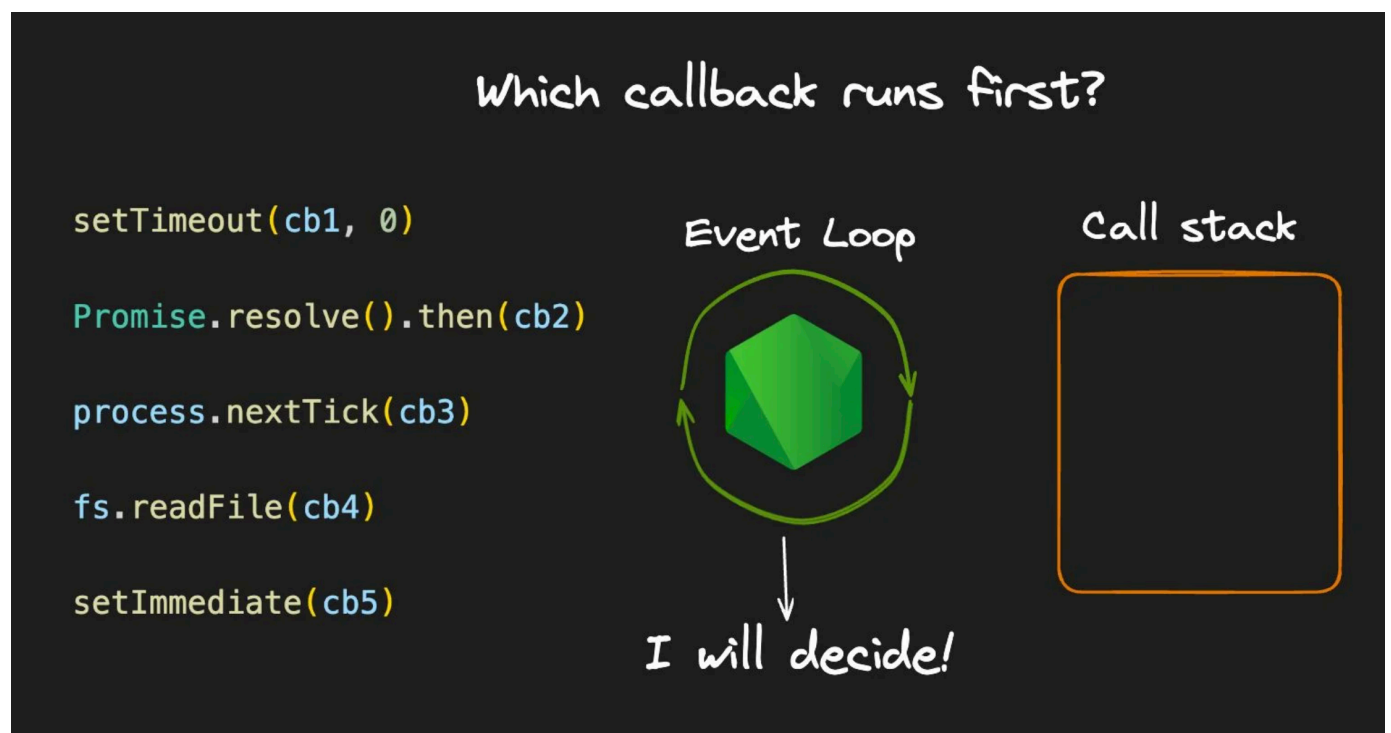
4. Libuv completes the task in the background
 5. When done, it tells JavaScript: "Hey, I'm done! Execute the callback"
-

The Event Loop

What is Event Loop?

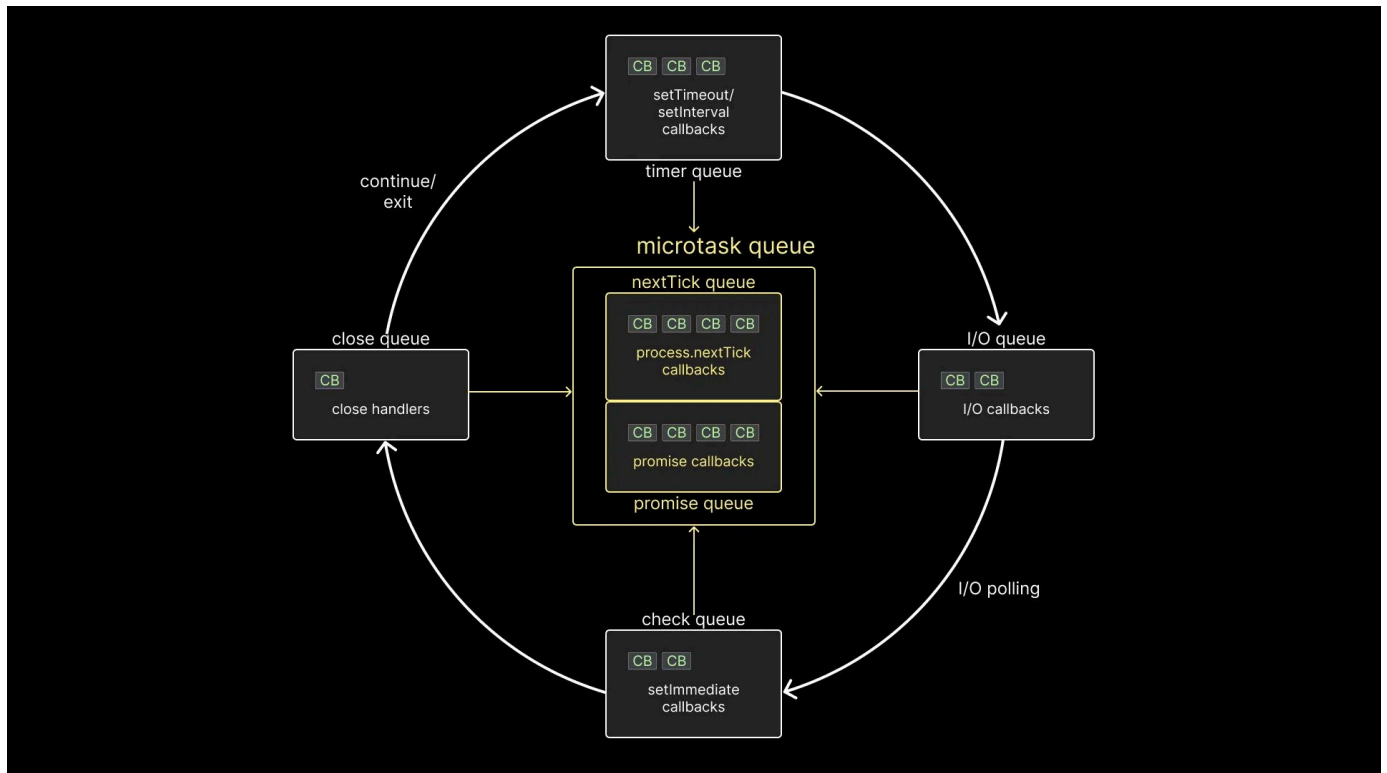
The event loop is like a **traffic controller** that manages when and in what order code should run. It continuously checks if there are tasks to execute.

Technically, the event loop is just a C program. But, you can think of it as a design pattern that orchestrates or coordinates the execution of synchronous and asynchronous code in Node.js. The event loop runs continuously as long as your Node.js application is up and running, handling multiple operations executing concurrently



The 6 Queues in Event Loop

Think of the event loop as having 6 different waiting lines (queues):



Execution Order - The Golden Rules

Rule #1: Synchronous code ALWAYS runs first. Event loop only starts when call stack is empty.

Rule #2: In the event loop, execution order is:

1. Execute all **nextTick** queue callbacks
2. Execute all **Promise** queue callbacks
3. Execute all **Timer** queue callbacks
4. After EACH timer callback → Execute microtask queues again
5. Execute all **I/O** queue callbacks
6. After EACH I/O callback → Execute microtask queues again
7. Execute all **Check** queue callbacks
8. After EACH check callback → Execute microtask queues again
9. Execute all **Close** queue callbacks
10. Execute microtask queues one last time

Rule #3: Microtask queues have the HIGHEST priority. They run after every single callback from other queues.

Code Examples

Example 1: Synchronous Execution

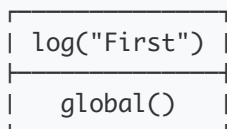
```
// Synchronous code
console.log("First");
console.log("Second");
console.log("Third");

// Output:
// First
// Second
// Third

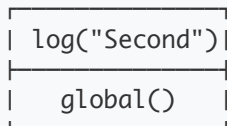
// Explanation:
// All code runs line by line in order
```

What happens in Call Stack:

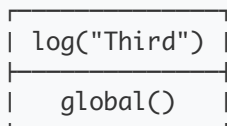
Time: 1ms



Time: 2ms



Time: 3ms



Example 2: Asynchronous Execution

```
const fs = require('fs');

console.log("First");

fs.readFile("file.txt", () => {
  console.log("Second");
});

console.log("Third");

// Output:
// First
// Third
// Second

// Why this order?
// 1. "First" logs immediately (synchronous)
// 2. fs.readFile starts but is offloaded to libuv
// 3. "Third" logs immediately (synchronous)
// 4. When file reading completes, "Second" logs (asynchronous)
```

Visualization:

```
Time: 1ms - Console log "First" ✓
Time: 2ms - fs.readFile() sent to libuv (goes to I/O queue)
Time: 3ms - Console log "Third" ✓
Time: 4ms - File reading completes in libuv
Time: 5ms - Call stack is empty, event loop runs
Time: 6ms - Callback from I/O queue executes
Time: 7ms - Console log "Second" ✓
```

Example 3: Understanding Queue Priority

```
console.log("Start");

setTimeout(() => {
  console.log("Timer 1");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise 1");
});

process.nextTick(() => {
  console.log("NextTick 1");
});

console.log("End");

// Output:
// Start
// End
// NextTick 1
// Promise 1
// Timer 1

// Explanation:
// 1. "Start" and "End" are synchronous - run immediately
// 2. Event loop starts after call stack is empty
// 3. nextTick queue has highest priority - runs first
// 4. Promise queue runs next
// 5. Timer queue runs last
```

Example 4: Complex Example with Multiple Queues

```
const fs = require('fs');

console.log("Start");

setTimeout(() => {
  console.log("Timer 1");
  process.nextTick(() => {
    console.log("NextTick inside Timer");
  });
}, 0);

fs.readFile("file.txt", () => {
  console.log("File read complete");
});

setImmediate(() => {
  console.log("Immediate 1");
});

Promise.resolve().then(() => {
  console.log("Promise 1");
});

process.nextTick(() => {
  console.log("NextTick 1");
});

console.log("End");

// Output:
// Start
// End
// NextTick 1
// Promise 1
// Timer 1
// NextTick inside Timer
// File read complete
// Immediate 1
```

Step-by-step breakdown:

1. **Synchronous code runs:** "Start" and "End" print
2. **Event loop starts:**
3. **Microtask queue (nextTick):** "NextTick 1" prints
4. **Microtask queue (Promise):** "Promise 1" prints
5. **Timer queue:** "Timer 1" prints
6. **Microtask queue again:** "NextTick inside Timer" prints
7. **I/O queue:** "File read complete" prints
8. **Check queue:** "Immediate 1" prints

Interview Questions & Answers

Q1: What is the Event Loop in Node.js? Explain in simple terms.

Answer:

The Event Loop is like a manager that continuously checks and executes code in Node.js. Think of it as a security guard who checks different waiting rooms in order.

Here's how it works:

- Node.js is single-threaded, meaning it can only do one thing at a time
- When you write asynchronous code (like reading a file), that task goes to the background
- The Event Loop keeps checking: "Is this task done? Is that task done?"
- When a task completes, the Event Loop brings its callback to execute

Simple analogy: Imagine you're cooking and you put rice in a cooker:

1. You don't stand and watch the rice cook (blocking)
 2. You do other tasks (washing vegetables, cutting onions)
 3. When the cooker beeps (task complete), you come back to check the rice
 4. The "you" in this story is the Event Loop!
-

Q2: What is the difference between `process.nextTick()` and `setImmediate()`?

Answer:

This is a tricky question many developers get confused about!

`process.nextTick()`:

- Executes in the **Microtask queue**
- Has the **HIGHEST priority**
- Runs **before** the event loop continues to the next phase
- Runs **after** the current operation completes

`setImmediate()`:

- Executes in the **Check queue**
- Runs **after** the I/O operations
- Designed to execute code in the **next iteration** of the event loop

Code Example:

```
setTimeout(() => {  
  console.log("Timeout");  
}, 0);  
  
setImmediate(() => {  
  console.log("Immediate");  
});  
  
process.nextTick(() => {  
  console.log("NextTick");  
});  
  
// Output:  
// NextTick  
// Timeout  
// Immediate
```

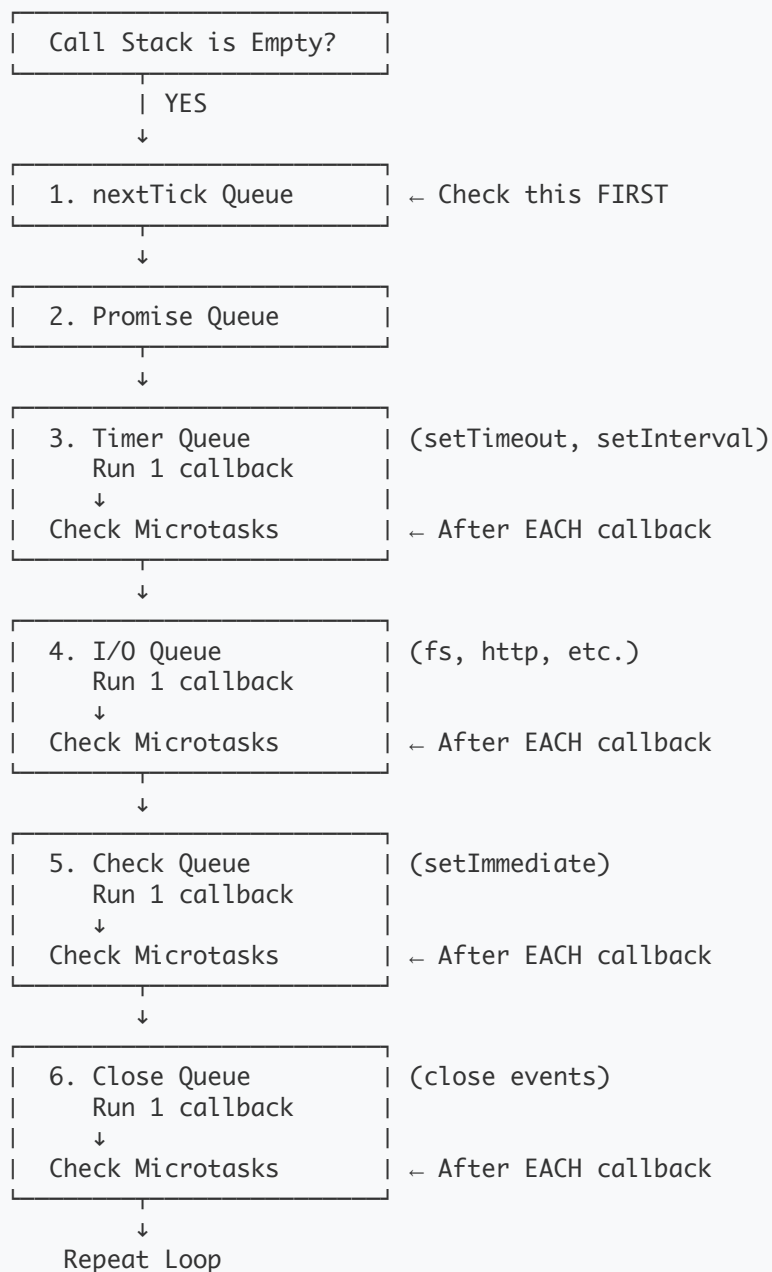
Memory tip:

- `nextTick` = "I need to run RIGHT NOW, next!"
 - `setImmediate` = "I can wait for the next cycle"
-

Q3: Explain the order of execution of different queues in the Event Loop.

Answer:

The Event Loop follows a specific order. Here's the complete flow:



Key Points:

1. Synchronous code ALWAYS runs first
2. Microtasks (nextTick + Promises) run after EVERY callback
3. The order is: Timer → I/O → Check → Close
4. Microtasks can starve the event loop if used excessively

Q4: What happens when the Call Stack is not empty?

Answer:

Simple rule: Event Loop NEVER interrupts JavaScript code execution.

When the call stack has code running:

- The Event Loop waits patiently

- No callbacks are executed
- No async operations are processed
- Everything waits until the call stack becomes empty

Example:

```
console.log("Start");

setTimeout(() => {
  console.log("Timer");
}, 0);

// Heavy blocking operation
for(let i = 0; i < 5000000000; i++) {
  // This blocks the call stack
}

console.log("End");

// Output:
// Start
// End (after long delay)
// Timer

// The setTimeout callback waits until the for-loop completes!
```

Why this design?

- JavaScript is single-threaded
- Only ONE task can run at a time on the call stack
- This prevents conflicts and race conditions

Q5: If setTimeout and fs.readFile both complete at the exact same time, which callback runs first?

Answer:

Timer callbacks run BEFORE I/O callbacks.

The event loop follows a strict order:

1. Microtasks (nextTick, Promises)
2. **Timer Queue** (setTimeout, setInterval)
3. **I/O Queue** (fs, http, network operations)
4. Check Queue (setImmediate)
5. Close Queue

Code Example:

```
const fs = require('fs');

setTimeout(() => {
  console.log("Timer callback");
}, 0);

fs.readFile("file.txt", () => {
  console.log("File read callback");
});

// Even if both complete at the exact same time:
// Output:
// Timer callback
// File read callback

// Timer has higher priority in the event loop!
```

Q6: What is Libuv and why is it important?

Answer:

Libuv is the magic behind Node.js asynchronous operations!

Libuv is a cross-platform open-source library written in C. In the Node.js runtime, its role is to provide support for handling asynchronous operations. Let's go over how this works.

What is Libuv?

- A C library (not JavaScript)
- Provides the Event Loop to Node.js
- Manages a thread pool for heavy operations
- Cross-platform (works on Windows, Linux, Mac)

Why it's important:

1. Makes Node.js non-blocking:

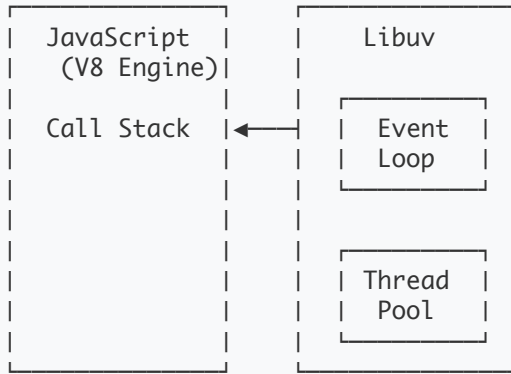
- JavaScript is single-threaded
- Libuv handles async tasks in the background
- Your JavaScript code keeps running without waiting

2. Provides the Event Loop:

- Manages all the queues (Timer, I/O, Check, etc.)
- Decides when to execute callbacks
- Coordinates synchronous and asynchronous code

3. Thread Pool:

- For tasks that the OS can't handle asynchronously
- Examples: file operations, DNS lookups, some crypto operations
- Default pool size: 4 threads (can be changed)

Visual Understanding:**Example:**

```

const fs = require('fs');

// This goes to Libuv
fs.readFile("file.txt", (err, data) => {
  console.log("File read complete!");
});

// JavaScript continues immediately
console.log("This runs while file is being read!");

// Libuv uses its thread pool to read the file
// When done, it notifies the event loop
// Event loop then executes the callback
  
```

Q7: Can you explain with code why Node.js is called "non-blocking"?**Answer:**

Non-blocking means the main thread doesn't wait for time-consuming operations.

Blocking Example (without Node.js async features):

```
// Imagine this is blocking code
function readFileBlocking() {
  // This takes 3 seconds
  let data = syncReadFile("bigfile.txt"); // Blocks for 3 seconds
  return data;
}

console.log("Start");
let data = readFileBlocking(); // Everything waits here for 3 seconds
console.log("Data:", data);
console.log("End");

// Output (with delays):
// Start
// ... 3 seconds wait ...
// Data: [file contents]
// End
```

Non-blocking Example (with Node.js):

```
const fs = require('fs');

console.log("Start");

// Non-blocking - doesn't wait!
fs.readFile("bigfile.txt", (err, data) => {
  console.log("Data:", data); // Runs later when ready
});

console.log("End");

// Output (immediate):
// Start
// End
// Data: [file contents] ← appears later

// The program doesn't wait!
// It continues executing while file is being read in background
```

Real-world benefit:

```
// Imagine a web server
const http = require('http');
const fs = require('fs');

http.createServer((req, res) => {
  // User 1 requests a big file
  fs.readFile("bigfile.txt", (err, data) => {
    res.end(data);
  });

  // User 2 can be served immediately!
  // Server doesn't wait for User 1's file to finish loading
}).listen(3000);

// This is why Node.js can handle 10,000+ concurrent connections!
```

Q8: What are Microtasks and why do they have the highest priority?

Answer:

Microtasks are special callbacks that need to run as soon as possible.

Two types of Microtasks:

1. **nextTick Queue:** `process.nextTick()` callbacks
2. **Promise Queue:** Promise `.then()` and `.catch()` callbacks

Why highest priority?

- They represent important, quick operations
- Usually used for cleanup, error handling, or critical updates
- Need to run before moving to next event loop phase

Example showing priority:

```
console.log("1. Synchronous");

setTimeout(() => {
  console.log("4. Timer");
}, 0);

Promise.resolve().then(() => {
  console.log("3. Promise");
});

process.nextTick(() => {
  console.log("2. NextTick");
});

// Output order:
// 1. Synchronous (runs immediately)
// 2. NextTick (highest priority microtask)
// 3. Promise (second priority microtask)
// 4. Timer (regular queue)
```

Important gotcha - Microtask starvation:

```
function recursiveNextTick() {
  console.log("Running");
  process.nextTick(recursiveNextTick); // ⚠️ Dangerous!
}

setTimeout(() => {
  console.log("I will never run!"); // This NEVER executes!
}, 0);

process.nextTick(recursiveNextTick);

// Problem: nextTick keeps adding itself back
// Event loop never moves to Timer queue
// This is called "microtask starvation"
```

Best practice:

- Use `process.nextTick()` for critical updates only
- Don't create recursive nextTick calls
- Prefer `setImmediate()` for less critical operations

Key Takeaways for Interviews

1. **JavaScript is single-threaded** - only one call stack, one thing at a time
2. **Event Loop manages async code** - it's like a traffic controller
3. **6 queues in Event Loop:**
 - Microtask (nextTick + Promise) ← Highest priority
 - Timer (setTimeout, setInterval)
 - I/O (fs, http)
 - Check (setImmediate)
 - Close (close events)
4. **Synchronous code ALWAYS runs first** - Event loop waits for call stack to be empty
5. **Libuv is the hero** - provides event loop and thread pool
6. **Microtasks run after every callback** - this is why they can "starve" the event loop
7. **Order matters:** Timer → I/O → Check → Close (with microtasks in between)

Quick Reference Chart

Feature	Description	Example
<code>setTimeout</code>	Timer queue, runs after delay	<code>setTimeout(() => {}, 0)</code>
<code>setImmediate</code>	Check queue, next iteration	<code>setImmediate(() => {})</code>
<code>process.nextTick</code>	Highest priority microtask	<code>process.nextTick(() => {})</code>
<code>Promise</code>	Microtask queue	<code>Promise.resolve().then()</code>
<code>fs.readFile</code>	I/O queue	<code>fs.readFile('file', cb)</code>
<code>http.request</code>	I/O queue	<code>http.get('url', cb)</code>

Practice Questions

Test your understanding:

1. Predict the output:

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
```

2. What's the order?

```
process.nextTick(() => console.log('A'));
setImmediate(() => console.log('B'));
setTimeout(() => console.log('C'), 0);
```

3. Why does this happen?

```
const fs = require('fs');

fs.readFile('file.txt', () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate'));
});
// immediate prints before timeout - Why?
```

Answers:

- 1, 4, 3, 2 (sync first, then promise, then timer)
- A, C, B (nextTick, timer, immediate)
- Inside I/O callback, `setImmediate` runs before `setTimeout` because Check queue is processed before going back to Timer queue

Remember: Understanding the Event Loop is crucial for building efficient Node.js applications! 🚀